

Husky Field Exploration

Task 4 Report: Future Directions, Improvements, and a First Research Paper

Atharva Kulkarni
athkul0601@gmail.com

26 September 2026

1 Scope and summary

This report addresses Task 4 of the assignment: directions for future work and improvements derived from the navigation framework built in Task 2, and from the limitations and performance observed during the Task 3 exploration deployments, together with an argument for a first research paper growing out of those improvements. Everything below is grounded in the implementation in the `husky-field-exploration` repository; parameter values quoted for the primary stack are the deployed defaults from its configuration files; backup-explorer values are the defaults declared in its Python node.

The overall conclusion is positive but specific. The Task 2/3 stack (RTAB-Map ICP SLAM, Nav2 planning and control, and a custom information-gain frontier explorer, on a VLP-16-equipped Clearpath Husky in Gazebo Fortress) carried out autonomous exploration of an unknown world. A separate grid-based coverage explorer was also implemented and attempted as a backup exploration strategy. The information-gain frontier explorer remained the primary method used for the final Task 3 deployment, while the coverage explorer was retained as a fallback and alternative heuristic. The weaknesses themselves concentrate in three places: the information-gain estimate is an occlusion-blind area count; the decision layer is greedy and sequential, spending up to seconds per cycle checking candidate goals one at a time; and the system was evaluated qualitatively (video), with no logged coverage, time, or failure metrics, so improvements cannot currently be measured. Section 5 turns each of these into a concrete improvement plan, and Section 6 argues that the first two, combined with the missing benchmark, form a coherent and feasible first research paper. Table 1 recaps the deployed system.

2 Literature basis

This work builds on established literature in frontier exploration, information-driven exploration, SLAM and skid-steer robotics. Frontier exploration selects destinations at the boundary between known free and unexplored space [1]. Information-based exploration evaluates candidate sensing actions according to their expected information or localization value [3, 4]. RTAB-Map provides an open-source framework for visual and LiDAR SLAM and long-term mapping [10]. For the Husky’s skid-steer platform, experimentally identified kinematics provide a stronger basis for motion estimation and control than assuming ideal differential-drive behaviour [9]. The present work keeps the SLAM and navigation stack fixed and investigates improvements to the exploration decision layer.

3 What Task 3 exposed: limitations of the deployed system

Task 3 ran the full stack in a $30\text{ m} \times 26\text{ m}$ unknown environment (`world/task3_explorer.sdf`) with the Husky spawned in a corner. The limitations below were identified from that deployment and from close reading of the implementation; they are ordered by their expected effect on exploration performance. Table 2 maps each to its root cause in the code and its practical effect.

3.1 The decision layer is greedy, sequential, and occlusion-blind

The explorer scores each candidate viewpoint c with

$$U(c) = I(c) - w d(c), \quad w = 12.0, \quad (1)$$

where $d(c)$ is the robot’s Euclidean distance to the candidate and $I(c)$ counts unknown cells inside a disc $D(c)$ of sensor radius $r_s = 4\text{ m}$ around the candidate, subsampled at a two-cell stride $\Delta = 2$:

$$I(c) = \sum_{(i,j) \in D(c)} \Delta^2 \mathbb{1}[m_{ij} = \text{unknown}], \quad (2)$$

m_{ij} being the occupancy value of grid cell (i, j) . Three problems follow. First, $I(c)$ is occlusion-blind: unknown area behind a wall counts exactly as unknown area in line of sight, so the utility systematically overvalues

Table 1: The deployed system at a glance.

Subsystem	Implementation	Key deployed parameters
Platform and sensing	Clearpath Husky (skid steer) in Gazebo Fortress; VLP-16 GPU LiDAR and IMU; <code>ros_gz_bridge</code>	LiDAR 10 Hz; IMU 50 Hz; cloud on <code>/velodyne_points</code>
Low-level control	<code>gz_ros2_control</code> diff-drive controller (<code>husky_velocity_controller</code>)	50 Hz odometry; <code>cmd_vel_unstamped</code> input
Task 1 controller	Python node: feed-forward + feedback unicycle tracking; polar pose regulation	50 Hz; saturation 0.6 m/s, 1.5 rad/s
SLAM	RTAB-Map, ICP registration on external wheel odometry, loop closure	2D grid, 5 cm cells; range 0.2–20 m; obstacle heights 0.05–1.8 m
Global planning	Nav2 NavFn (Dijkstra)	tolerance 0.65 m; <code>allow_unknown: true</code>
Local control	Nav2 DWB + velocity smoother; recovery behaviors	10 Hz; max 0.6 m/s, 1.5 rad/s; spin/backup/wait
Costmaps	Voxel layer (local) + static/obstacle layers (global) + inflation	5 cm resolution; inflation 0.55 m (local), 0.60 m (global)
Exploration (Task 3)	<code>information_gain_explorer</code> node: frontier detection, information-gain ranking, Nav2 feasibility checks, goal watchdog	2 s decision cycle; 45 s goal timeout; max 20 planner candidates

candidates whose sensor disc is largely occluded. Second, the two terms in Eq. (1) have inconsistent units (a raw cell count versus meters), which makes the trade-off weight w map- and resolution-dependent and forces hand tuning. Third, the choice is greedy: the best-scoring candidate is executed and the cycle repeats, with no ordering of the remaining frontiers, the classic source of back-and-forth travel in frontier exploration [1].

Feasibility filtering compounds the latency. Candidates are checked one at a time through Nav2’s `ComputePathToPose` action, up to `max_planner_candidates` = 20 per decision cycle, each check a global-costmap search followed by a Python path-clearance scan over the returned poses. While this runs, the robot holds its previous goal and the next decision cycle is gated behind a 2 s timer, so the worst case leaves the robot idle through several seconds of sequential checks before it receives its next viewpoint. The blacklist mechanism (failed and visited goal lists, 0.75 m exclusion radius) never decays: a viewpoint that failed once, for instance because a loop closure shifted the map under an in-flight goal, is excluded forever, together with a disc of frontier around it.

3.2 Backup coverage explorer

A second exploration strategy was implemented as a fallback to the primary information-gain frontier explorer. The node `fieldrobo/coverage_explorer.py` deliberately removes both frontier detection and information-gain scoring. Instead, it maintains a set of visited known-free occupancy-grid cells, groups remaining known-free cells into coarse spatial regions, generates safe candidate viewpoints within those regions, and ranks the regions primarily by distance from the robot. The implementation therefore provides a substantially simpler coverage-oriented heuristic alongside the primary frontier/information-gain method.

The coverage explorer was attempted as a backup in case the information-gain frontier decision layer proved unreliable during integration. Its candidate goals are still checked with Nav2’s `ComputePathToPose` before a goal is dispatched, and successful navigation updates the visited-cell set; failed goals are placed in a local blacklist. The implementation also reports the fraction of currently known free cells marked as visited, providing a useful diagnostic for a coverage-based strategy.

The two approaches therefore serve different roles in the project. The primary explorer asks *which frontier viewpoint should reveal the most useful new information for the travel cost?*; the backup explorer asks *which currently known free regions have not yet been visited, and which of them can be reached safely?*. The backup remains available as a fallback and as a possible future heuristic baseline without being presented as the final Task 3 exploration method.

3.3 State estimation and mapping

RTAB-Map registers scans with ICP on top of wheel odometry from the diff-drive controller. The IMU is bridged into ROS but unused, and the Husky is a skid-steer vehicle whose wheel odometry is least accurate precisely during the yaw-heavy maneuvers that exploration induces; short-term odometry error feeds directly into ICP correspondence quality. Loop closures do correct the graph, but they arrive as map-to-odom transform jumps: goals expressed in the map frame, and a global costmap built moments earlier, can become briefly inconsistent with the world, which the 45 s watchdog then penalizes as a goal failure. The 2D projection (3D gridding disabled, obstacles between 0.05 m and 1.8 m, range capped at 20 m) assumes flat ground; negative

Table 2: Limitations observed in Task 3, their root causes, and their practical effects.

Limitation	Root cause in the implementation	Practical effect
Gain ignores occlusion	Disc count of unknown cells, $r_s = 4$ m; no ray casting	Overvalues viewpoints behind walls; poor goal ranking
Mixed units in utility	Cell-count gain minus $12.0 \times$ distance in meters	Weight requires retuning per map and resolution
Greedy one-step selection	Best-scoring candidate executed each cycle	Back-and-forth travel and revisits
Sequential feasibility checks	Up to 20 <code>ComputePathToPose</code> calls, one at a time, per cycle	Seconds of decision latency while the robot waits
Full-grid scans in Python	<code>get_frontiers()</code> visits every cell each cycle (312,000 cells in the $30\text{ m} \times 26\text{ m}$ world at 5 cm)	CPU cost grows with map area; caps scalability
Permanent blacklists	failed/visited goal lists, 0.75 m exclusion, no decay	Valid regions can become permanently unreachable
Map-frame goal instability	Loop-closure map-to-odom jumps under in-flight goals	Watchdog misfires; good goals blacklisted
Odometry drift in turns	Wheel odometry only; IMU bridged but unused	Skid-steer yaw error degrades ICP registration
Flat 2D world assumption	2D grid; obstacles 0.05–1.8 m; 20 m range cap	No slopes, negative obstacles, or overhangs
Diff-drive model of a skid steer	Fixed wheel separation; unicycle laws; finite-difference reference velocities	Tracking error on arcs; slow trajectories misclassified
No quantitative evaluation	Video-only results; one world; no baselines	Improvements cannot be measured

obstacles, slopes, and overhangs, the normal case in field robotics, are invisible to it.

3.4 Navigation and control

Nav2’s DWB tracks the NavFn path using velocity sampling tuned for differential-drive motion, while the Husky is skid steer. The local and global inflation radii (0.55 m and 0.60 m) also reduce the set of passages considered navigable. Recovery behaviours can escape local minima, but repeated in-place rotations can increase wheel-odometry error. The Task 1 controller uses feed-forward/feedback unicycle tracking and polar pose regulation; this is suitable for the nominal model, but its fixed wheel-separation parameter is only an approximation for skid-steer motion. Experimental skid-steer studies motivate explicit treatment of slip and identified kinematics [9]. Reference velocities are estimated from consecutive pose references, introducing additional filtering lag.

3.5 The evaluation gap

The most consequential limitation is methodological: Task 3 produced a video, not a dataset. No coverage-over-time curve, path length, planner-failure count, decision latency, or localization error was logged; a single world was used, and no baseline (for example an `explore_lite`-style nearest-frontier policy) was run for comparison. As a result, the value of the information-gain heuristic relative to simpler or more sophisticated alternatives is unknown, and any future improvement is unverifiable. This gap is also, conveniently, the entry point for the research paper argued in Section 6.

4 Simulation challenges and problems encountered

Integration exposed several practical issues before the final Task 3 deployment. The VLP-16 GPU LiDAR had to be bridged from Gazebo into ROS 2 with the correct topic, frame, message type and QoS; the final cloud was published at approximately 10 Hz and consumed by RTAB-Map and Nav2 costmaps. The simulated sensor also has a 0.3 m minimum range, and near-field returns were observed at this boundary, highlighting the importance of sensor mounting geometry.

RTAB-Map required consistent connections between wheel odometry, the point cloud and TF. During exploration the occupancy map changed continuously, so frontier candidates could become invalid after new observations or loop-closure corrections. A geometrically valid frontier was also not necessarily a successful navigation goal: Nav2 sometimes rejected candidates at the global-planning stage, while DWB could report that no valid local trajectories were available. The explorer therefore added `ComputePathToPose` feasibility checks, path-clearance filtering and a failed-goal blacklist.

The remaining challenge was computational. Frontier detection and candidate evaluation operate on a growing occupancy grid, while feasibility checks are performed sequentially. This can delay the next exploration decision. These integration problems directly motivated the future work below: faster candidate evaluation, navigation-aware scoring, improved state estimation and quantitative benchmarking.

5 Future work and improvements

The improvements below are grouped by subsystem and ordered by expected return on effort inside this codebase. Each item names the change, why the Task 3 evidence motivates it, and what it would replace. The proposed implementation sequence is given at the end of this section.

5.1 A principled information objective

Replace the disc count of Eq. (2) with an occlusion-aware expected information gain. Casting rays from the candidate through the current occupancy grid (Bresenham on the 5 cm grid is sufficient at this scale) counts only unknown cells actually reachable by the sensor model:

$$I_{\text{ray}}(c) = \sum_{r \in \mathcal{R}(c)} \sum_{p \in r} \Delta^2 \mathbb{I}[m_p = \text{unknown} \wedge r \text{ reaches } p], \quad (3)$$

where $\mathcal{R}(c)$ is the ray set covering the sensor field of view from c . The probabilistic formulation is the mutual information between the candidate observation z_c and the map M , i.e. the expected entropy reduction $I(c) = H(M) - H(M | z_c)$, in the spirit of information-theoretic exploration [3, 4]. For a binary-occupancy grid with unknown cells at $p = 0.5$, the per-cell entropy $H(m_{ij}) = -p \log p - (1-p) \log(1-p)$ is maximal, which is why counting unknown cells is already a usable first-order approximation; the ray-cast form simply removes the part of that count the sensor cannot realize.

To remove the unit mismatch of Eq. (1), the terms should be normalized over the current candidate set $\mathcal{C}$ (for instance by min-max scaling, $\hat{I}(c) = I(c) / \max_{c'} I(c')$, $\hat{d}(c) = d(c) / \max_{c'} d(c')$), and a normalized clearance term $\hat{R}(c)$ can absorb the risk awareness that the current path-clearance check provides only after the fact:

$$\hat{U}(c) = \hat{I}(c) - \lambda_d \hat{d}(c) + \lambda_r \hat{R}(c). \quad (4)$$

The same machinery extends to active SLAM, where candidates are scored on map entropy and localization uncertainty together (for example D-optimal criteria on the pose-graph covariance) [11]. Because the candidate set changes slowly relative to the 2 s decision cycle, gains can be cached and updated incrementally.

5.2 Viewpoint sequencing instead of one-step greed

Order the currently reachable frontier viewpoints rather than repeatedly picking a single winner. A nearest-neighbor + 2-opt TSP tour over the top- k candidates, replanned every cycle (a receding horizon), retains reactivity while suppressing the oscillation that pure greediness causes. A more formal route treats viewpoint selection as monotone submodular maximization: if the set function $f(S)$ capturing the total information collected by visiting viewpoint set S is normalized, monotone, and submodular, then lazy greedy evaluation achieves at least

$$f(S_{\text{greedy}}) \geq \left(1 - \frac{1}{e}\right) \max_{|S| \leq k} f(S) \quad (5)$$

under a cardinality budget k [5, 6], and with caching typically evaluates far fewer candidates than the exhaustive sort performed today.

5.3 Making the decision layer real-time

Three mechanical fixes, each independent: (i) vectorize frontier detection and clustering with NumPy/SciPy (`scipy.ndimage.label` with an 8-connected structure) instead of Python sets, since the full 312,000-cell grid of the test world is scanned every cycle today; (ii) replace sequential `ComputePathToPose` calls with concurrent action requests (a multi-threaded executor), or with a single flood-fill reachability check on the costmap, reserving the Nav2 planner for the finally selected goal; and (iii) maintain frontiers incrementally from map updates rather than rescanning. Together these should cut decision latency from seconds to well under one second, which matters because the robot is effectively idle while decisions are made.

5.4 Estimation: fuse what is already there

The IMU topic already crosses the Gazebo bridge. Feeding wheel and IMU odometry through the EKF provided by `robot_localization` is a configuration change, not a research project, and directly attacks the yaw-drift-during-turns problem. Going further, LiDAR odometry (RTAB-Map’s `icp_odometry`, or a KISS-ICP-class method) would decouple the odometry source from skid-steer slip. The map-jump problem needs a policy rather than a bigger hammer: track in-flight goals in the odom frame between map updates, and on a loop-closure event re-validate the active goal instead of letting the watchdog blacklist it.

5.5 Navigation upgrades

DWB can be replaced by Nav2’s MPPI controller, whose sampled rollouts handle diff-drive-like platforms and tight spaces better than DWB’s critic ensemble, and NavFn by the Smac Hybrid-A* planner for kinodynamically feasible global paths [7, 8]. A decaying blacklist (exponential forgetting, or re-eligibility when

the underlying cells change) removes the permanent-exclusion failure mode of Section 3.1. Doorway-width passages need a justified inflation radius or a dedicated narrow-passage mode, since the current radii exclude them by construction.

5.6 Control: model the skid steer

The unicycle tracking and pose-regulation laws used in Task 1 are correct for the modeled plant; the plant is the approximation. An extended kinematic model with longitudinal and lateral instantaneous centers of rotation (Mandow-style identification [9]) replaces the fixed-wheel-separation abstraction with, in the body frame,

$$\omega = \frac{v_R - v_L}{Y_R - Y_L}, \quad v_x = \frac{v_R Y_L - v_L Y_R}{Y_L - Y_R}, \quad v_y = -\omega X_{IC}, \quad (6)$$

where v_L, v_R are the left/right track speeds, Y_L, Y_R the lateral ICR offsets, and X_{IC} the longitudinal ICR offset, all identified experimentally and gain-scheduled on speed. This would cut tracking error on aggressive arcs. Publishing reference velocities explicitly (or a proper trajectory message) removes the finite-difference reference-velocity estimate and its lag, and the moving-versus-stationary switching threshold disappears with it. A Lyapunov analysis of the composite law (tracking + regulation + saturation) is a publishable fragment in itself.

5.7 An evaluation harness, before anything else

Before any algorithmic change: a headless Gazebo benchmark that logs coverage fraction over time, path length, decision latency, recovery counts, and (against Gazebo ground truth) odometry drift, run over multiple worlds and seeds with results written to file. Without it, Sections 5.1–5.6 cannot be credited; with it, the ablation study of Section 6 exists almost for free.

5.8 The longer arc: 3D, multi-robot, field

Later extensions include elevation-aware traversability and negative-obstacle detection, multi-Husky map merging and frontier allocation, and sim-to-real validation on a physical Husky.

A practical sequence is to first build the evaluation harness and reduce decision latency, then test occlusion-aware gain and navigation upgrades, and finally study viewpoint sequencing and skid-steer modelling across multiple environments.

6 The first research paper

6.1 The gap

The current system is representative of a wider situation: the ROS 2 exploration ecosystem runs on mature, well-engineered planning stacks (Nav2, SLAM packages such as RTAB-Map [10]) whose decision layers are either decades-old heuristics (nearest frontier [1], in the spirit of `explore_lite`) or research prototypes published against bespoke stacks that others cannot run. Published information-gain and active-SLAM methods [3, 4, 6] are rarely evaluated against the baselines practitioners actually deploy, on a common modern middleware, with the computational cost of the gain computation reported alongside its benefit. That reproducible, ablation-driven middle ground is the opening, and this project already contains both sides of it: a working modern stack and a decision layer whose every component can be swapped and measured.

6.2 Proposed paper and contributions

Working title: *Occlusion-Aware, Feasibility-Filtered Frontier Sequencing for Rapid Ground-Robot Exploration*. Target length 6–8 pages, well matched to IEEE RA-L. The claimed contributions would be:

1. An occlusion-aware, sensor-model-aware information gain computed by grid ray casting, embedded in a normalized multi-objective utility (gain, travel cost, clearance risk), replacing the disc-count heuristic of Eq. (2).
2. A receding-horizon viewpoint sequencing layer (a TSP tour over candidate viewpoints, or lazy-greedy submodular selection carrying its $(1 - 1/e)$ approximation guarantee of Eq. (5)), together with concurrent Nav2 feasibility filtering that keeps decision latency sub-second.
3. A reproducible benchmark on an open ROS 2 Humble / Gazebo Fortress / RTAB-Map / Nav2 stack: multiple worlds, seeds, and baselines (nearest frontier; the current disc-count system as an ablation; and, where useful, the implemented coverage heuristic), reporting coverage time, path length, decision latency, and planner failures, plus a real-Husky demonstration.

The scope is deliberately confined to the decision layer: SLAM, costmaps, and low-level control stay fixed, so every comparison isolates exactly the component under test. This is also what makes the paper feasible as a first paper: every method component is an incremental modification of the existing repository, and every baseline is a configuration variant of the same stack.

Table 3: Benchmark metrics for the proposed study.

Metric	Definition	What it tests
Time to 90% coverage	Simulated time from spawn until the free-space coverage curve crosses 90%	H1 and overall method comparison (primary metric)
Total path length	Odometry-integrated distance travelled over the run	H2: sequencing versus one-step greed
Decision latency	Time from decision-cycle start to goal dispatch	Feasibility layer and sub-second claim
Recoveries and failures	Count of BT recoveries, watchdog timeouts, and blacklisted goals	Robustness and the map-jump problem
Odometry drift	Trajectory error versus Gazebo ground-truth poses	SLAM coupling; context for all other metrics
Gain computation cost	Wall-clock time of the utility evaluation per cycle	Whether ray-cast gain is affordable online

6.3 Hypotheses and experimental design

H1: occlusion-aware gain reaches a fixed coverage fraction faster than disc-count gain, with the largest margin in high-occlusion worlds (rooms, long walls). **H2:** receding-horizon sequencing reduces total path length versus one-step greedy selection at an equal gain computation budget. **H3:** the utility’s components contribute separately and in a stable rank order across worlds, as shown by ablation.

The design: at least four Gazebo world classes (open, corridor-heavy, room-structured, cluttered; the Task 3 world is one of them), ten seeds per cell, fixed compute per run, paired statistical tests, and all runs launched from a single script so the study is one command to reproduce. Table 3 lists the metrics.

6.4 Justification for choice of first paper

It grows directly out of working code, so the risk is concentrated in the experiments rather than in the engineering; it has a clear, falsifiable question (does principled, feasibility-aware goal selection buy measurable exploration performance, and at what computational price?); and it produces an artifact the community lacks (a reproducible benchmark on a standard modern stack). Each hypothesis has a named ablation, so a reviewer can trace every claim to a specific comparison; and the failure modes are interesting either way, since a negative result on occlusion-aware gain in open worlds is itself evidence about when the extra machinery pays off.

6.5 Risks and counterarguments

Novelty is the main risk: frontier exploration is a mature field and each ingredient exists somewhere. The defense is the composite contribution, namely the approximation guarantee where submodular selection applies, the feasibility-aware integration with a standard stack, and above all the reproducible benchmark with ablations. A second risk is simulation-only evidence; a real Husky run, even a limited one, plus releasing worlds, configurations, and results as an open artifact, is the mitigation. The third risk, that ray-cast gain is too slow online, is an experimental question the benchmark itself answers, and vectorized ray casting on a 5 cm grid is cheap next to a Nav2 global plan.

7 Code and video artefacts

The complete implementation, configuration files, Gazebo world, launch files and experimental artefacts are available at <https://github.com/Ath0601/husky-field-exploration>. The repository also contains the attempted `coverage_explorer` fallback, exposed as the ROS 2 executable `coverage_explorer`; the documented Task 3 launch continues to use the primary `frontier_explorer`. Task 1 and Task 3 demonstration videos are provided through the repository rather than embedded in the PDF. The Task 3 video shows autonomous exploration, incremental map construction and navigation between exploration goals, ending with the completed map.

8 AI tool usage (Task 4)

An LLM-based AI assistant (Sarvam AI) was used for repository/code analysis, report structuring and drafting assistance. Technical claims were checked against the implementation and configuration files; the experimental interpretation, prioritization and final research direction were decided and edited by the author.

References

- [1] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *Proc. IEEE Int. Symp. on Computational Intelligence in Robotics and Automation (CIRA)*, 1997.
- [2] Y. Kanayama, Y. Kimura, F. Miyazaki, and T. Noguchi, “A stable tracking control method for an autonomous mobile

- robot,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 1990.
- [3] F. Bourgault, A. Makarenko, S. B. Williams, B. Grocholsky, and H. F. Durrant-Whyte, “Information based adaptive robotic exploration,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2002.
 - [4] C. Stachniss, *Robotic Mapping and Exploration*, Springer Tracts in Advanced Robotics, vol. 55, 2009.
 - [5] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, “An analysis of approximations for maximizing submodular set functions—I,” *Mathematical Programming*, vol. 14, 1978.
 - [6] A. Singh, A. Krause, C. Guestrin, and W. J. Kaiser, “Efficient informative sensing using multiple robots,” *Journal of Artificial Intelligence Research*, vol. 34, 2009.
 - [7] S. Macenski, F. Martin, R. White, and J. Gines Clavero, “The marathon 2: A navigation system,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2020.
 - [8] G. Williams, P. Drews, B. Goldfain, J. M. Reh, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2017.
 - [9] A. Mandow, J. L. Martinez, J. Morales, J. L. Blanco, A. Garcia-Cerezo, and J. Gonzalez, “Experimental kinematics for wheeled skid-steer mobile robots,” in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2007.
 - [10] M. Labbe and F. Michaud, “RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation,” *Journal of Field Robotics*, vol. 36, no. 2, 2019.
 - [11] H. Carrillo, Y. Latif, J. Neira, and J. A. Castellanos, “On the comparison of uncertainty criteria for active SLAM,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2015.